

Executable Knowledge Lifecycles as Dynamical Control in Agent Systems:

A Typed-Compilation Study in Synthetic Causal Environments

Yongjing Gao

v0.2.6-exp.1-final

Abstract

In a synthetic causal environment, a model-external typed compiler separates candidate knowledge from trusted knowledge. Compiler output is constrained to an unscored `CANDIDATE`; only an epistemic validation plane may assign continuous confidence and admit execution. Across controlled interventions, executing correct knowledge improves robustness, executing incorrect knowledge causes directional harm, and isolating expired knowledge has no effect. Forced activation of expired knowledge restores the harm. These results support a narrow systems claim: an executable knowledge lifecycle acts as a dynamical control structure in an agent system. They do not establish causal discovery or broad real-world validity. The semantic study contains 18 unique cases (3 preregistered core cases and 15 cross-domain extensions) and 390 decoding events; decode repetitions are reported as repetitions, not independent semantic samples.

1 Problem Statement

Agent systems commonly pass natural-language “knowledge” directly into planning or learning components. This collapses parsing, validation, admission, activation, and execution into one probabilistic channel. We study a stricter lifecycle:

$$K_{\text{NL}} \xrightarrow{\text{compile}} K_{\text{IR}}^{\text{candidate}} \xrightarrow{\text{validate}} K_{\text{IR}}^{\text{trusted}} \xrightarrow{\text{activate}} \mathcal{L}(K). \quad (1)$$

The central question is not whether a language model “knows” a causal fact. It is whether system boundaries cause correct, wrong, stale, and adversarially injected knowledge to produce distinguishable and auditable dynamics.

2 System and Threat Model

The compiler emits one JSON object conforming to a Draft-07 causal-claim schema. Markdown, missing fields, unknown fields, invalid control metadata, and

prefixed or suffixed prose fail outside the model. Governance metadata is sealed as `status=CANDIDATE` and `claim_confidence=null`. An epistemic data plane alone converts lineage count, sample size, and consistency into a numeric confidence.

We separate two hashes. *SemanticHash* covers cause, effect, scope, time window, intervention, and effect specification. *EnvelopeHash* covers identifier, version, status, provenance, and confidence. This prevents governance serialization from being confused with executable causal semantics.

The threat model includes causal-direction inversion, temporal drift, indirect prompt injection, hallucinated provenance, conflicting claims, expired-claim activation bypass, and ungrounded defaults induced by long-context distractors.

3 Executable Causal Regularization

For model f_θ , active claim K , non-causal feature set $N(K)$, and validated confidence c_K , training minimizes

$$\mathcal{L}(K) = \mathcal{L}_{\text{task}} + \lambda \mathbf{1}[K \text{ active}] c_K \mathbb{E}_x \left[\sum_{j \in N(K)} \left\| \frac{\partial f_\theta(x)}{\partial x_j} \right\|_2^2 \right]. \quad (2)$$

The synthetic environment contains one structural feature X_1 and two training-environment shortcuts X_2, X_3 . Shortcut correlations reverse out of distribution.

We define E_{OOD} operationally as the held-out shortcut-intervention MSE with X_1 fixed. It is the same raw observation reported as E_{inv} ; it is not a general-purpose measure of distribution shift. We report

$$\Delta E_{\text{OOD}} = E_{\text{base}} - E_K, \quad (3)$$

$$R_{\text{OOD}} = \frac{\Delta E_{\text{OOD}}}{E_{\text{base}} + 10^{-12}}. \quad (4)$$

Raw predictive OOD MSE is retained separately.

4 Experiments

Typed compilation (1A). The core cohort has 3 handcrafted semantic cases with 30 decode replicates each. The extension adds 15 cases across quantitative trading, advertising technology, clinical and industrial safety, supply chain, cybersecurity, energy, manufacturing, credit, logistics, agriculture, recommendation, telecommunications, education technology, and insurance, with 20 replicates per case. Thus 390 events do not imply 390 independent semantic cases.

We preregister a *severe semantic error* as any decoding event that reverses causal direction, changes required temporal or intervention semantics, fabricates protected provenance, violates frozen governance metadata, or disagrees with Gold IR on a required executable field.

Execution conformance (1B). Compiled natural language and native Gold IR are compared by SemanticHash and EnvelopeHash. When semantic hashes match, downstream (E_{inv} , E_{sens}) must be exactly equal.

Knowledge execution (1C). No-K, correct, wrong, partial, stale, and fault-activated stale conditions share data, initialization, and decoding seeds.

Stress tests. We scan the preregistered grids

$$\lambda \in \{0, .01, .03, .1, .3, 1, 3, 10\},$$
$$c_K \in \{.001, .005, .01, .025, .05, .1, .2\}.$$

Every point uses 30 paired seeds and a paired Student-*t* 95% interval. Lambda is selected on validation MSE without inspecting held-out OOD outcomes.

5 Results

The direct natural-language path has a severe semantic error rate of 76.7% overall (74.4% in the core cohort and 77.3% in the extension), while typed compilation has 0.0%. SemanticHash agreement is 100.0% (390/390), and EnvelopeHash agreement is 100.0%. These are system-level results: deterministic source anchors and model-external normalization are part of the compiler and should not be interpreted as raw language-model accuracy.

As shown in Table 1, correct knowledge removes nearly all shortcut-intervention sensitivity. Wrong knowledge amplifies error in the opposite direction. Partial knowledge produces an intermediate outcome. Inactive stale knowledge is exactly equivalent to No-K under shared initialization, while activation bypass produces material harm. Low-confidence wrong knowledge remains harmful but its mean harm increases monotonically with confidence. Table 2, Panel A reports the manually authored domain labels.

6 Limitations and Validity

The execution environment is synthetic, linear, and deliberately small. The cross-domain cases broaden vocabulary and business context, but they remain manually authored templates with explicit variable labels. Decode replicates measure stochastic stability, not population-level semantic diversity. The benchmark evaluates compilation fidelity against frozen Gold IR; it does not validate the truth of those claims. $E_{\text{OOD}} \approx E_{\text{inv}}$ is an operational identity in this experiment, not evidence that invariance universally measures OOD generalization. Confidence scoring is a reference control law rather than a calibrated empirical estimator. No clinical, financial, or safety deployment claim follows from these results.

7 Reproducibility and Provenance

Schemas, compiler, loss operator, frozen cases, seeds, raw metrics, confidence intervals, and failure flags are versioned together. Paper tables are generated directly from JSON artifacts. The compilation artifact records case counts and decode-replicate counts separately. All model inference was local; untrusted raw generations are not persisted in the formal artifact.

Table 2, Panel B gives the release locator state and the deterministic evidence bundle. The current freeze is not a Git worktree, so repository URL, tag, and commit remain explicitly unavailable unless injected by the publishing environment; no synthetic locator is substituted.

8 Conclusion

Within the stated synthetic scope, the evidence supports a control-structure interpretation of executable knowledge. Correct execution improves behavior; wrong execution damages it; lifecycle isolation prevents stale knowledge from acting; bypassing isolation restores harm. The engineering result is the separation of probabilistic parsing from deterministic admission and activation, not a claim of autonomous causal understanding.

References

- [1] J. Pearl, *Causality: Models, Reasoning, and Inference*, 2nd ed., Cambridge University Press, 2009.
- [2] J. Peters, D. Janzing, and B. Schölkopf, *Elements of Causal Inference*, MIT Press, 2017.
- [3] M. Arjovsky et al., “Invariant Risk Minimization,” arXiv:1907.02893, 2019.

Table 1: Executable-knowledge outcomes. ΔE_{OOD} is absolute shortcut-intervention error reduction; R_{OOD} is its normalized counterpart. Predictive OOD MSE remains separately available in the artifact.

Condition	ΔE_{OOD}	R_{OOD}	E_{inv}	E_{sens}	C_{train}	C_{infer}
No K	0.0000	0.0000	1.6330	1.0392	1.0000	1.0000
Correct K	1.6330	1.0000	6.35e-08	0.0008	1.8788	1.0000
Wrong K	-4.1422	-2.5365	5.7751	2.0116	1.6949	1.0000
Partial K	1.3570	0.8310	0.2760	0.3924	1.6480	1.0000
Stale K	0.0000	0.0000	1.6330	1.0392	1.0032	1.0000
Stale K, fault-injected active	-5.2758	-3.2308	6.9088	2.0079	1.8925	1.0000

Table 2: Semantic coverage and release provenance. Domain labels are not independent statistical samples; null external locators are disclosed rather than replaced with synthetic repository metadata.

Panel A: benchmark coverage

Domain	Cases
advertising_technology	1
agriculture	1
clinical_safety	1
credit_risk	1
cybersecurity	1
education_technology	1
energy_grid	1
industrial_safety	1
insurance	1
logistics	1
manufacturing	1
quantitative_trading	1
recommender_systems	1
supply_chain	1
synthetic_core	3
telecommunications	1

Panel B: release and evidence locators

Release field	Frozen value
Release ID	v0.2.6-exp.1-final
Repository URL	git@github.com:JingyongGao/ executable-knowledge-lifecycles.git
VCS tag	<i>not assigned</i>
Source commit	<i>not available</i>
Evidence bundle	artifacts/v0.2.6-exp.1-final-evidence.zip
External bundle	<i>not assigned</i>
URL	
Bundle SHA-256	ff70923d21cf0cd794b52aacb4fc4e5570275af4919871d7 d0032412a6493725